

GUIÓN DE EXPOSICIÓN

MLflow Jedi

La Cámara de los Experimentos Perdidos

Un guion oral, humanizado y completo para acompañar la presentación y la demostración de la app.

MLFLOW	APP	CIERRE
≈ 8 min	≈ 8 min	≈ 1,5 min

FRASE ANCLA Scikit-learn entrena. MLflow registra. El Consejo compara y decide.

Cómo usar este documento

- No intentes memorizarlo palabra por palabra. **Lee primero cada frase ancla** y cuenta el resto con tus palabras.
- Las cajas azules indican **qué señalar, cuándo cambiar de diapositiva o cuándo entrar en la app**.
- El texto principal está escrito para sonar hablado: incluye pausas, ejemplos y frases de transición.
- Duración objetivo: **unos 17 minutos y medio**, con margen para respirar o responder una interrupción.

Antes de empezar · preparación mínima

- Deja la presentación abierta en la diapositiva 1 y la app abierta en otra pestaña.
- Si vas a enseñar la app en directo, lleva preparada una **sesión de prueba**, un equipo ya dentro y, si es posible, tres runs completadas. Así no dependes del tiempo de entrenamiento.
- Ten localizada una matriz de confusión y el informe JSON para que el comparador no se quede en una tabla de decimales.
- Si falla Internet, continúa con las diapositivas 6 a 10: el guion no depende de que la demo funcione.

Diapositiva 1 · Apertura

Tiempo orientativo: 0:45

EN PANTALLA Mira al público antes de hablar. No leas el subtítulo completo; úsalo solo como contexto.

Hoy os voy a presentar **La Cámara de los Experimentos Perdidos**, un proyecto para aprender MLflow de una forma bastante práctica y, sobre todo, sin separar la parte técnica de la toma de decisiones.

La idea utiliza el dataset Digits de scikit-learn, modelos Random Forest y una aplicación en Streamlit. Toda la estética está inspirada en la Biblioteca Jedi, pero la narrativa no está puesta solo para decorar: la usamos como una especie de mapa mental para entender qué guarda MLflow y por qué lo guarda.

Voy a dividir la explicación en dos partes. Primero quiero dejar claro **qué problema resuelve MLflow y cómo funciona el tracking**. Después os enseño la aplicación y cómo convierte esos conceptos en una actividad de equipo.

FRASE ANCLA Primero entendemos la memoria del experimento; después vemos cómo la practicamos en la app.

Diapositiva 2 · El problema: entrenar no es suficiente

Tiempo orientativo: 1:15

EN PANTALLA Señala los tres nombres de archivo y deja una pausa breve para que se reconozca la situación.

Quería empezar por una escena que creo que nos ha pasado a casi todos: **modelo_final**, **modelo_final_bueno**, **modelo_final_ahora_si**. Al principio parece una tontería, pero en cuanto hacemos varios experimentos aparece un problema serio.

Podemos tener un modelo guardado y, aun así, no saber con qué datos se entrenó, qué hiperparámetros tenía, qué métricas obtuvo, qué clases confundía o por qué decidimos quedarnos con él. Es decir, conservamos el resultado final, pero perdemos la historia que permite entenderlo y repetirlo.

Y para mí esa es la idea de partida del proyecto: **experimentar mucho no es el problema**. De hecho, queremos probar alternativas. El problema aparece cuando cada prueba queda aislada, sin contexto y sin evidencias comparables.

FRASE ANCLA El problema no es experimentar muchas veces; el problema es perder la historia de cada experimento.

Diapositiva 3 · Qué es MLflow Tracking

Tiempo orientativo: 2:00

EN PANTALLA Recorre visualmente los cuatro bloques: decisiones previas, resultados, archivos y reproducibilidad.

MLflow Tracking es, básicamente, **un sistema de registro y organización de ejecuciones de machine learning**. No sustituye a scikit-learn, no entrena por nosotros y tampoco elige el mejor modelo. Su trabajo es conservar de forma ordenada lo que ocurrió en cada ejecución.

Antes del entrenamiento tenemos decisiones previas: por ejemplo, usar 50, 100 o 300 árboles, limitar o no la profundidad, fijar el tamaño del conjunto de prueba y guardar el `random_state`. Eso son los **parameters**, porque describen la configuración con la que entramos al entrenamiento.

Después de entrenar obtenemos resultados numéricos: accuracy, precision, recall o F1. Eso son las **metrics**. La diferencia parece sencilla, pero es esencial: un parámetro expresa una decisión tomada antes; una métrica expresa un resultado observado después.

Luego están los **artifacts**, que son archivos con más contexto que una cifra aislada. En este proyecto guardamos una matriz de confusión en PNG y un classification report en JSON. La accuracy puede decirme que el modelo acierta alrededor del 96 %, pero la matriz me permite ver dónde se equivoca y qué dígitos está confundiendo.

También registramos el **modelo entrenado**. En nuestro caso se guarda mediante la integración de MLflow con scikit-learn, junto con una firma de entradas y salidas y un pequeño ejemplo de entrada. Así no conservamos solo el número final: conservamos un objeto que se puede cargar y cuyo contrato de datos queda documentado.

Todo esto aporta trazabilidad. Si una run tiene un identificador, sus parámetros, métricas, artefactos, modelo y etiquetas, otra persona puede comparar lo que hicimos y acercarse mucho más a reproducirlo.

Digo ‘acercarse’ porque MLflow ayuda muchísimo, pero una reproducción perfecta también exige controlar datos, código, dependencias y entorno.

FRASE ANCLA MLflow no entrena ni decide: conserva las evidencias necesarias para comparar y reproducir.

Diapositiva 4 · La Biblioteca del Templo Jedi

Tiempo orientativo: 2:00

EN PANTALLA Baja por la tabla de arriba abajo. No leas el código completo; explica la función de cada pieza.

Aquí es donde entra la analogía Jedi. La **Biblioteca** representa MLflow Tracking completo: el lugar donde queda organizado el conocimiento. Dentro de esa biblioteca, una **campaña** equivale a un Experiment, es decir, el contenedor que agrupa ejecuciones relacionadas.

Cada **misión** es una Run. Una run representa una ejecución concreta: un entrenamiento con una configuración y unos resultados determinados. Por eso, si entreno tres Random Forest distintos, necesito tres runs, aunque estén dentro del mismo Experiment.

La **estrategia** son los Parameters: lo que decidimos antes de entrenar. El **informe del Consejo** son las Metrics: lo que observamos al evaluar. El **holocrón** es un Artifact: una evidencia que conserva información rica, como una imagen o un informe. Y el **Jedi entrenado** representa el Model que podemos guardar y recuperar.

En el código, el orden sería muy parecido a este: primero indicamos dónde se va a guardar el tracking; después seleccionamos o creamos el Experiment; abrimos una Run con start_run; y, dentro de esa run, registramos parámetros, métricas, artefactos y modelo.

En esta aplicación el tracking se guarda en una base SQLite de MLflow y los artefactos quedan en el directorio de datos. Además, cada Experiment incorpora el código de sesión y cada run lleva etiquetas de equipo y sesión. Eso permite que una actividad con varios grupos no termine mezclada en un único bloque imposible de rastrear.

La última fila es intencionada: el **Consejo Jedi no es una función de MLflow**. Es el equipo humano que interpreta las evidencias. MLflow organiza la información, pero el criterio sigue siendo nuestro.

FRASE ANCLA Experiment agrupa; Run representa una ejecución; Parameters describen; Metrics evalúan; Artifacts explican; Model conserva.

Diapositiva 5 · El flujo de un experimento

Tiempo orientativo: 1:45

EN PANTALLA Sigue el diagrama con el dedo o el cursor: *datos → entrenamiento → evaluación → registro → comparación → decisión.*

Si juntamos las piezas, este es el recorrido completo. Partimos del dataset Digits, preparamos los datos y entrenamos un Random Forest con scikit-learn. Después generamos predicciones y calculamos las métricas.

A continuación MLflow registra la configuración y los resultados de esa ejecución. En nuestro proyecto son **cinco parámetros, cuatro métricas globales más dos recalls por clase, dos artefactos y un modelo**. Repetimos el proceso con tres configuraciones y obtenemos tres runs comparables.

La comparación viene después. Y esto es importante: no elegimos automáticamente la run con la accuracy más alta. Miramos el contexto, el coste, el comportamiento por clase y las evidencias. La decisión final puede cambiar según el problema que estemos intentando resolver.

Con esto cierro la parte conceptual. Ya sabemos qué memoria conserva MLflow. Ahora voy a enseñar cómo la aplicación obliga a utilizar esa memoria para argumentar una decisión.

FRASE ANCLA Los datos y scikit-learn producen el modelo; MLflow conserva su historia; las personas interpretan la evidencia.

Diapositiva 6 · La aplicación

Tiempo orientativo: 1:15

CAMBIO A LA APP Si haces demo, abre ahora la pestaña de Streamlit. Si no, mantén esta diapositiva y describe el mismo recorrido.

La aplicación se llama igual que el proyecto y está construida con Streamlit. Hay dos entradas:

Profesorado y Alumnado. El profesorado crea una sesión y obtiene un código con formato JEDI-XXXX. El alumnado no crea cuentas: introduce ese código y elige uno de diez nombres de equipo disponibles.

El trabajo está pensado para grupos de tres. El **Navegante** controla el recorrido, el **Escriba** introduce el código y el **Auditor** comprueba que lo que se registra corresponde realmente a parámetros, métricas o evidencias. Estos roles ayudan a que no sea una actividad de una sola persona delante del teclado.

Desde el panel docente se puede ver en qué cámara está cada equipo, sus intentos, pistas, puntuación y fase. También se puede enviar un aviso global, desbloquear un progreso, reiniciar un equipo, reasignar un holocrón y exportar los resultados en JSON o CSV.

FRASE ANCLA La app convierte el tracking en un recorrido colaborativo, no en una lista de funciones para memorizar.

Diapositiva 7 · Las seis cámaras y el marcador

Tiempo orientativo: 1:45

EN PANTALLA En la app, entra en Mapa y luego en Cámara actual. Enseña una teoría breve, el editor y el botón de pista.

Cada sesión selecciona **seis preguntas de un banco de cincuenta**. Hay una pregunta por cada concepto esencial: Experiment, Run, Parameters, Metrics, Artifacts y Model. Así cambia la actividad entre sesiones, pero nunca perdemos la cobertura del contenido.

Los retos no son todos iguales. Algunos son de elección múltiple; otros piden completar código, detectar una llamada equivocada, corregirla o clasificar una información. Cada cámara incluye una explicación corta, el contexto necesario, un editor y tres pistas progresivas.

La puntuación hace visible el progreso sin convertir el error en un bloqueo. Resolver una cámara suma 20 puntos. Una pista resta 5. Hay cinco oportunidades y, si el equipo falla las cinco, aparece la solución, se aplican 10 puntos de penalización y puede continuar.

Esa decisión pedagógica me parece importante: equivocarse tiene una consecuencia, pero nadie se queda encerrado en la primera parte y pierde toda la práctica posterior.

FRASE ANCLA La variación evita memorizar respuestas; el avance asistido evita que un error detenga todo el aprendizaje.

Diapositiva 8 · Seguridad y tres misiones

Tiempo orientativo: 2:15

EN PANTALLA En Laboratorio, muestra las tarjetas de Tatooine, Coruscant y Mustafar y abre el editor de una misión.

Esta parte tenía un reto técnico claro: quería que el alumnado escribiera sintaxis real de MLflow, pero no podía ejecutar directamente cualquier texto escrito en un editor web.

La solución fue separar **validación** y **ejecución**. El código del alumnado se convierte en un árbol sintáctico, un AST. El validador revisa qué llamadas aparecen, si hay imports, si se usa un bloque with cuando corresponde, si se están mezclando parámetros y métricas y si están presentes todas las piezas exigidas.

Si la estructura es válida, la aplicación **ignora ese texto para la ejecución** y lanza una plantilla interna controlada. Esa plantilla carga Digits, hace una división estratificada con un 20 % de test y random_state 42, entrena el Random Forest, calcula las métricas, genera los dos artefactos y registra una run real en MLflow. Así conservamos el valor de escribir código sin abrir ejecución arbitraria en el servidor.

Las misiones aumentan la autonomía. **Tatooine** está guiada y usa 50 árboles con profundidad máxima 8. **Coruscant** tiene 100 árboles, profundidad 10 y un sabotaje: parámetros y métricas están intercambiados y hay que corregirlos. **Mustafar** pide el tracking completo casi desde cero, con 300 árboles y sin límite de profundidad.

El recordatorio 5-4-2-1 resume el mínimo: cinco parámetros, cuatro métricas globales, dos artefactos y un modelo. Además, la ejecución real añade recall del dígito 1 y del dígito 8 como métricas diagnósticas, porque luego serán útiles para tomar decisiones que no dependan solo de la accuracy.

FRASE ANCLA El alumnado escribe y razona el tracking; la aplicación valida la estructura; solo una plantilla interna segura entrena y registra.

Diapositiva 9 · Comparador de runs y Orden 66

Tiempo orientativo: 2:15

EN PANTALLA *En Comparador, señala primero parámetros y métricas; después abre una matriz o el JSON. Pulsa Orden 66 solo si la sesión de demo está preparada.*

Cuando terminan las tres misiones, ya no tenemos un único modelo: tenemos tres runs con distinta complejidad. Los resultados orientativos muestran que Mustafar obtiene la mejor accuracy y F1 global, y también mejora mucho el recall del dígito 1. Pero el recall del 8 se mantiene alrededor de 0,8571 en las tres.

Ese dato sirve para demostrar que **una cifra global puede ocultar un comportamiento concreto**. Si el problema exige reconocer ochos, Mustafar no resuelve automáticamente la limitación aunque tenga más árboles y una accuracy mayor. Por eso el comparador permite abrir la matriz de confusión y el classification report: necesitamos ver el patrón de error, no solo el promedio.

Después aparece 'Ejecuten Orden 66'. No cambia las runs ni fabrica resultados nuevos. Asigna al equipo uno de siete criterios de decisión: por ejemplo, recursos limitados, auditoría, comunicación a un público no técnico, recall del 1, recall del 8 o preparación para producción.

Y aquí está el objetivo final del proyecto: con los mismos datos, dos equipos pueden recomendar runs distintas si su criterio y su evidencia están bien explicados. El equipo con recursos limitados puede preferir una mejora más modesta y barata; el equipo centrado en rescatar unos priorizará recall_1; el de auditoría mirará parámetros, artefactos, modelo y run_id.

El holocrón cambia la pregunta que hacemos a los resultados. **No cambia los resultados**. Esa diferencia obliga a argumentar y evita que la actividad termine en 'gana el decimal más alto'.

FRASE ANCLA La accuracy resume; los artefactos explican; el criterio de negocio decide qué evidencia importa.

Diapositiva 10 · Defensa ante el Consejo y cierre

Tiempo orientativo: 1:30

REGRESO A DIAPOSITIVAS *Vuelve a la presentación. Haz una pausa antes de la frase final grande.*

El recorrido termina con una defensa de 45 a 60 segundos. Cada equipo tiene que elegir una run, compararla con otra, citar al menos dos métricas, indicar qué artefacto consultó, explicar un hallazgo, reconocer una limitación y terminar con una recomendación.

La plantilla les da estructura, pero no les da la respuesta: ‘Elegimos esta run porque... Consultamos este artefacto y observamos... Descartamos esta otra por... Nuestra decisión tiene esta limitación... y recomendamos...’. Cuando la sellan, el profesorado puede revisarla desde su panel.

Si tuviera que resumir todo el proyecto en una sola idea, sería esta: **un buen experimento no es solo un modelo con una métrica alta; es una decisión que otra persona puede entender, revisar y discutir con evidencias.**

La Biblioteca del Templo Jedi impedía que el conocimiento se perdiera. MLflow hace lo mismo con nuestros experimentos. Scikit-learn entrena. MLflow registra. Y el Consejo, es decir, nosotros, compara y decide.

FRASE ANCLA **Un experimento útil no termina al entrenar: termina cuando podemos explicar y defender la decisión.**

Control del tiempo

Si necesitas dejarlo en 15 minutos

- Diapositiva 3: resume Parameters, Metrics, Artifacts y Model en una sola frase cada uno.
- Diapositiva 4: explica la analogía sin leer las llamadas de código.
- Diapositiva 7: menciona el banco de 50, las seis categorías y el desbloqueo tras cinco fallos; omite el detalle completo de puntuación.
- Diapositiva 9: abre solo un artefacto y explica un único holocrón.

Si puedes acercarte a 20 minutos

- En la diapositiva 3, añade un ejemplo completo: `n_estimators=100` como parámetro, `accuracy=0,9611` como métrica y `matriz_confusion.png` como artefacto.
- En la diapositiva 6, enseña brevemente el panel del profesorado y el código JEDI-XXXX.
- En la diapositiva 8, muestra el sabotaje de Coruscant y pregunta al público qué llamada está mal.
- En la diapositiva 9, compara Mustafar y Coruscant y abre la fila del dígito 8 en el informe JSON.

Plan B si la app no carga

Puedes decir: **‘La aplicación está desplegada, pero para no depender de la conexión voy a enseñaros el recorrido desde las diapositivas. Lo importante es el flujo, porque la ejecución y el registro son exactamente los mismos.’** Después continúa con las diapositivas 6 a 10 sin disculparte más.

Posibles preguntas y respuestas cortas

Pregunta	Respuesta breve
¿MLflow entrena el modelo?	No. Aquí entrena scikit-learn. MLflow registra la ejecución y conserva sus evidencias.
¿Por qué SQLite?	Porque para una actividad autocontenida permite guardar los metadatos de tracking sin desplegar un servidor adicional. Para escalar o tener varios nodos usaría un backend externo.
¿El código del alumnado se ejecuta?	No. Se analiza con AST; si es válido, se ejecuta una plantilla interna conocida.
¿Por qué no elegir siempre la mayor accuracy?	Porque puede ocultar errores por clase y no incorpora coste, trazabilidad ni criterio de uso.
¿Es totalmente reproducible?	MLflow mejora mucho la reproducibilidad, pero también hay que versionar datos, código, dependencias y entorno.
¿Qué se guarda exactamente por run?	Cinco parámetros, seis métricas en total, dos artefactos, el modelo, su firma, un ejemplo de entrada y etiquetas de equipo y sesión.

TARJETA DE RESCATE

Diez ideas para hablar sin papel

1. Problema: perdemos la historia de cada experimento.
2. MLflow registra y organiza; no entrena ni decide.
3. Experiment agrupa runs relacionadas.
4. Parameters = decisiones previas; Metrics = resultados posteriores.
5. Artifacts explican los errores; Model conserva el objeto entrenado.
6. La app trabaja con seis cámaras y tres misiones progresivas.
7. El código se valida con AST y nunca se ejecuta directamente.
8. Se registran 5 parámetros, 4 métricas globales + 2 recalls, 2 artefactos y 1 modelo.
9. Orden 66 cambia el criterio de decisión, no los resultados.
10. Scikit-learn entrena; MLflow registra; el Consejo compara y decide.

FRASE ANCLA Si te quedas en blanco, vuelve al flujo: entrenar → registrar → comparar → decidir.